

Formal Methods Update Meeting 2026

Edited by

Inzemamul Haque¹, T. V. H. Prathamesh², and Aalok Thakkar³

¹ Krea University, Sri City, India inzemamul.haque@krea.edu.in

² Krea University, Sri City, India prathamesh.turaga@krea.edu.in

³ Ashoka University, Sonipat, India thakkar@ashoka.edu.in

Abstract

The Formal Methods (FM) Update Meeting is an annual gathering of the Indian formal methods community, bringing together researchers and practitioners working on the use of formal methods in hardware and software design, verification and synthesis, model checking, theorem proving, and theoretical computer science. FM Update 2026 was held at Krea University, Sri City, on 2–3 July 2026, with tutorials on 1 July and 4 July. This report collects abstracts of all talks and tutorials presented at the meeting.

Seminar 01.–04. July, 2026 – <https://fm-update.github.io/>

2012 ACM Subject Classification Theory of computation → Logic and verification; Theory of computation → Automated reasoning; Theory of computation → Formal languages and automata theory

Keywords and phrases Formal methods, verification, model checking, automata, logic, program synthesis, theorem proving

1 Executive Summary

Inzemamul Haque (Krea University, IN, inzemamul.haque@krea.edu.in)

T. V. H. Prathamesh (Krea University, IN, prathamesh.turaga@krea.edu.in)

Aalok Thakkar (Ashoka University, IN, thakkar@ashoka.edu.in)

Welcome to the Formal Methods Update Meeting 2026. The FM Update series has, over the years, become the annual meeting point of the Indian formal methods community: an informal venue where researchers and students working across verification, model checking, theorem proving, automata, logic, and program synthesis come together to share what they have been thinking about, arguing about, and building. We were delighted to host the 2026 edition at Krea University, Sri City, and to bring the community together on the Krea campus for four days of talks, tutorials, and conversation.

FM Update 2026 was hosted at Krea University from 1 to 4 July 2026, and was co-located with the Indian School of Logic and Applications 2026. The meeting followed the informal, community-driven format that has defined the series: contributed talks by researchers based in India and abroad, together with tutorials aimed at students and newcomers to the field. The meeting opened with a tutorial on reactive synthesis by Shibashis Guha, and closed with a hands-on tutorial on applied formal methods in cryptography delivered by the Society for Electronic Transactions and Security (SETS), Chennai. Twenty-four contributed talks were presented over the main days.

The programme covered a broad slice of current work in formal methods, including

- automata theory and logic, with talks on generalized timed automata, set automata for two-variable logic on data words, regular expressions over countable words, circuit complexity of regular languages, edit distance of transducers, and effectively propositional logic with function symbols;
- verification of concurrent, probabilistic, and reactive systems, including model checking via weighted model counting, sure-almost-sure window mean payoff in MDPs, POMDP policy synthesis, and QBF proof systems for Skolem/Herbrand extraction;
- program verification and analysis, including IC3 for software, non-termination via recurrence-seeking tests, incorrectness proofs with underapproximation invariants, verification modulo tested library contracts, and liquid tree automata for refinement-typed synthesis;
- interactive theorem proving and autoformalization, including certified regular-expression hardware in Rocq, bit-vector reasoning in Isabelle/HOL, and combinator-code-generation approaches to autoformalization;
- security and cryptography, including intruder theories for the symbolic analysis of protocols, and end-to-end verifiable election protocols;
- hardware verification, including symbolic trajectory evaluation for error-correction algorithms.

This volume collects the abstracts of every talk delivered at the meeting, in the order they were presented, together with the list of participants. We hope it serves both as a record of the meeting and as a useful reference for readers looking to find out who is working on what in Indian formal methods today.

We gratefully acknowledge the generous support of our sponsors, without whom the meeting would not have been possible: the Indian Association for Research in Computing Science (IARCS), Ashoka University, Krea University, and TCS Research. We also thank the

staff at Krea University for their hospitality, the speakers and tutorial presenters for their contributions, and everyone who travelled to Sri City to make FM Update 2026 what it was.

2 Table of Contents

Executive Summary

<i>Inzemamul Haque, T. V. H. Prathamesh, and Aalok Thakkar</i>	2
--	---

Tutorials

Tutorial on Reactive Synthesis <i>Shibashis Guha</i>	6
Tutorial on Applied Formal Methods in Cryptography <i>SETS</i>	6

Talks

A Modal Approach Towards Substitutions <i>Sujata Ghosh</i>	6
Intruder Theories <i>S. P. Suresh</i>	7
Verification Modulo Tested Library Contracts <i>Abhishek Vijay Uppar</i>	7
Liquid Tree Automata, Tree Automata for Refinement Types <i>Ashish Mishra</i>	8
Communication Complexity of Automata-Theoretic Decision Problems <i>Abhishek De</i>	8
Recurrence Seeking Tests for Non-termination <i>Ravindra Metta</i>	8
Autoformalization by Combinator Code Generation <i>Siddhartha Gadgil</i>	9
Certified Regular Expression Matcher in Rocq <i>Piyush Kurur</i>	9
Bridging Bit-Vectors and Natural Numbers in Isabelle/HOL <i>Shweta Rajiv</i>	9
Verifying Error Correction Coding Algorithms using Symbolic Trajectory Evaluation <i>Ramit Das</i>	10
Incorrectness Proofs with Underapproximation Invariants <i>Gourav Takhar</i>	10
Formal Verification of End-to-End Verifiable Election Protocols <i>Subodh Sharma</i>	11
Edit Distance of Finite State Transducers <i>Aiswarya C.</i>	11
Proof Systems for QBF Synthesis: Extracting Skolem and Herbrand Functions <i>Supratik Chakraborty</i>	11
Generalized Timed Automata <i>B. Srivathsan</i>	12

Sure-Almost-Sure and Sure-Limit-Sure Window Mean Payoff in Markov Decision Processes <i>Pranshu Gaba</i>	12
Decision Tree Ensembles: A Sweet Spot between Machine Learning and Formal Methods <i>S. Akshay</i>	13
Understanding the IC3 verification algorithm <i>Deepak D'Souza</i>	13
Scaling Probabilistic Model Checking with Model Counting <i>Arijit Shaw</i>	13
Circuit Complexity of Regular Languages <i>Amaldev Manuel</i>	13
Set Automata and Limits of Decidability of Two-Variable Logic on Data Words <i>Rishal S. P.</i>	14
Regular Expressions over Countable Words <i>Sreejith A. V.</i>	14
Synthesizing POMDP Policies: Sampling meets Model-checking via Learning <i>Sayan Mukherjee</i>	15
Deciding Effectively Propositional Logic with Function Symbols <i>Abhisekh Sankaran</i>	15
Participants	17

3 Tutorials

3.1 Tutorial on Reactive Synthesis

Shibashis Guha (TIFR Mumbai, IN, shibashis@tifr.res.in)

Reactive synthesis is the problem of automatically constructing a system that interacts continuously with an environment while satisfying a formal specification. Instead of verifying a manually designed controller, synthesis aims to generate a correct-by-construction implementation directly from the specification itself. A central framework for this area is synthesis from Linear Temporal Logic (LTL), where desired behaviors are expressed using temporal operators describing how the system should evolve over infinite executions. The origins of the field trace back to the seminal “Church synthesis problem,” posed by Alonzo Church in the 1950s, which asked whether one can automatically construct a machine satisfying a logical specification against all possible inputs from an adversarial environment. Church’s question established the deep connection between logic, automata, and infinite games that still underpins modern reactive synthesis. Since then, LTL synthesis has become a foundational topic in formal methods, with applications ranging from hardware design and communication protocols to robotics and autonomous systems, while also driving major advances in automata theory, game theory, and algorithmic verification.

3.2 Tutorial on Applied Formal Methods in Cryptography

Society for Electronic Transactions and Security (SETS Chennai, IN)

This training aims to provide hands-on experience in formal verification methods for cryptographic systems and secure communication mechanisms. The sessions will introduce the foundations of formal verification and demonstrate how mathematical reasoning is used to establish strong security guarantees in adversarial environments. Participants will understand why conventional testing alone is insufficient for validating security-critical systems against malicious behavior and complex execution scenarios.

The training will cover symbolic, computational and implementation-level verification approaches for analyzing properties such as confidentiality, authentication, integrity, correctness, and memory safety. Through practical demonstrations and guided examples, attendees will gain exposure to attacker modeling, automated verification techniques, proof construction and secure implementation analysis.

4 Talks

4.1 A Modal Approach Towards Substitutions

Sujata Ghosh (Indian Statistical Institute, IN, sujata@isichennai.res.in)

Joint work of Ghosh, Sujata; Li, Dazhu; Liu, Fenrong; Tu, Yaxin

Substitutions play a crucial role in a wide range of contexts, from analyzing the dynamics of social opinions and conducting mathematical computations to engaging in game-theoretical analysis. For many situations, considering one-step substitutions is often adequate. Yet,

for more complex cases, iterative substitutions become indispensable. In this talk, we will describe logical frameworks that model both single-step and iterative substitutions. We will explore a number of properties of these logics, including their expressive strength, Hilbert-style proof systems, and satisfiability problems. We will also establish connections between our proposed frameworks and relevant existing ones in the literature.

4.2 Intruder Theories

S. P. Suresh (Chennai Mathematical Institute, IN, spsuresh@cmi.ac.in)

Joint work of Suresh, S. P.; Ramanujam, R.; Sundararajan, Vaishnavi

The Dolev-Yao model is the primary abstraction used in the symbolic analysis of cryptographic protocols. In this model, cryptographic operations are abstracted as operators in a term algebra and rules in a formal derivation system on messages. Special properties of some cryptographic operations like XOR, homomorphic encryption etc. are modelled as equation (or rewrite) theories, called “intruder theories” in the literature. In this lecture, we introduce the example of XOR, study its equation theory and how it can be incorporated into the derivation system. We also study the verification problem for protocols that use XOR, and we study a further extension to the protocol model that communicates assertions as well as terms.

4.3 Verification Modulo Tested Library Contracts

Abhishek Vijay Uppar (Indian Institute of Science, IN)

We propose a lightweight verification technique for client programs that use complex data-structure libraries. The basic idea is the following: given a client program P that uses a library L , and some correctness assertions in P , we find adequate program annotations in the client program (including loop invariants and pre-post contracts for the called library methods), that constitute a valid Floyd-Hoare-style proof that the program satisfies its assertions. The key departure from standard proofs is that the library contracts are not formally verified, but only *tested*. Hence the name of the approach: *Verification Modulo Tested Library Contracts* (VMTLC).

We also propose a technique for automatically synthesizing such VMTLC proofs based on an Implication-Counter-Example (ICE) learning framework. We implement this synthesis technique in a tool called Dualis that uses a variety of ICE-learners with a fuzzing-based tester in the loop, and evaluate it on a collection of client programs. Our results show that Dualis is able to find VMTLC proofs of several correct programs that are out of reach of state-of-the-art automated verification tools, while flagging several incorrect programs that pass testing.

4.4 Liquid Tree Automata, Tree Automata for Refinement Types

Ashish Mishra (IIT Hyderabad, IN, mishraashish@iith.ac.in)

Component-based synthesis (CBS) generates loop-free programs from library components to satisfy logical queries. While expressive specifications and precise queries simplify the solution space, they make finding feasible execution paths significantly more difficult for traditional CBS procedures. As constraints become more exact, the search must navigate an increasingly sparse space of valid paths. We address this challenge by reasoning about logical similarities between exploration paths. We consider library methods equipped with refinement-type specifications, which enrich base types with logical qualifiers to precisely constrain the value space. To efficiently explore this space, we introduce Liquid Tree Automata (LTA), a novel tree automata variant whose construction is driven by refinement typing rules. LTAs leverage subtyping constraints to identify and eagerly merge semantically similar states during search. This merging avoids redundant exploration of equivalent paths, significantly improving synthesis efficiency. We implement this approach in a tool called Hegel. Our evaluation demonstrates that Hegel synthesizes solutions to complex queries that are beyond the reach of existing state-of-the-art tools.

4.5 Communication Complexity of Automata-Theoretic Decision Problems

Abhishek De (Krea University, IN)

Joint work of De, Abhishek; Ghosh, Prantar

Decision problems, such as membership, emptiness, universality, and language equivalence, play a central role in automata theory. In a joint work with Prantar Ghosh, I looked at the (deterministic, nondeterministic, and randomised) communication complexity of such problems for regular languages when the input is given in the form of (deterministic or nondeterministic) finite automata. We consider two standard variants of the two-player communication model: (i) where each player gets an automaton and they solve comparison-based problems on the corresponding languages, and (ii) where an input automaton is split between two players and they want to determine certain properties of the automaton. We obtain tight (up to logarithmic factors) bounds for most problems: our upper bounds are deterministic, and lower bounds are randomised, almost settling the complexity of these problems.

4.6 Recurrence Seeking Tests for Non-termination

Ravindra Metta (TCS Research, IN)

Non-termination is a fundamental problem in program verification that is highly relevant in industrial software. Infinite-loop vulnerabilities continue to be reported in widely deployed software systems, including cloud and application infrastructure, where they can lead to denial-of-service, resource exhaustion, loss of responsiveness, and violation of timing requirements. Detecting such behaviour is challenging because it may depend on hard-to-reach control-flow

paths, long execution prefixes, or specific input patterns. We present DIRNT, a non-termination (NT) checking technique that introduces recurrence-seeking tests: tests designed to steer execution towards recurrent behaviour and detect NT when recurrence is observed during concrete execution. Unlike symbolic approaches, which derive NT evidence primarily through symbolic reasoning, DIRNT uses symbolic reasoning only to construct tests that guide execution towards recurrent behaviour. Unlike execution-based approaches, DIRNT systematically generates such tests with recurrence as the explicit objective, rather than relying on unguided runs. We implemented DIRNT for C programs and evaluated it on SV-COMP NT benchmarks and on non-terminating programs derived from open-source software, where it detects non-termination in 832 of 849 benchmarks ($\approx 98\%$) with an average analysis time of 7.36 seconds per benchmark, outperforming existing techniques in both effectiveness and efficiency under identical resource limits.

4.7 Autoformalization by Combinator Code Generation

Siddhartha Gadgil (Indian Institute of Science, IN, gadgil@iisc.ac.in)

Autoformalization is the translation of mathematics in natural language into a formal language that is machine-verifiable. In our system LeanAide, we introduce and use an autoformalization framework based on Combinator Code Generation, where we combine handlers for different mathematical constructs in a manner analogous to Combinator Parsing. We will discuss this approach and its various advantages, including extensibility and human readability of the generated code.

4.8 Certified Regular Expression Matcher in Rocq

Piyush Kurur (IIT Palakkad, IN, ppk@iitpkd.ac.in)

The talk is about designing certified hardware, i.e. hardware with a formal proof of correctness, in Rocq. We present a library for generating certified hardware for regular expression matching. The user gives a regular expression encoded using our eDSL in Rocq, and the hardware matcher is obtained using the `synthesize` function of our library. We prove the correctness of the `synthesize` function: the language of the input regular expression is the same as the language accepted by the generated hardware. This means the hardware is verified end-to-end with no particular effort from the user (push-button certified matcher). Experimental results show that the matcher is very efficient even when hardly any effort has gone into optimization other than choosing the right algorithm (Ken Thompson's algorithm). The code is available from the public repository at <https://gitlab.com/piyush-kurur/regexp-hardware>.

4.9 Bridging Bit-Vectors and Natural Numbers in Isabelle/HOL

Shweta Rajiv (The University of Iowa, US, shweta-rajiv@uiowa.edu)

Bit-vectors are a core tool in software and hardware verification, and modern solvers handle fixed-width bit-vectors very well using techniques like bit blasting. However, these approaches struggle with large bit widths and do not generalize to parametric bit-vectors, where the

size is symbolic and reasoning must work uniformly for all widths. In this talk, I present a mechanized translation from parametric bit-vectors to natural numbers in Isabelle/HOL. The approach builds on prior work on int blasting, encoding bit-vector operations as arithmetic over naturals with explicit bounds. We show that this translation preserves semantics for a wide range of operations, including arithmetic, logical, and bit-level constructs. The formalization is developed in two styles: a shallow embedding that supports automated rewriting of bit-vector goals into natural number formulas, and a deep embedding that establishes a global correctness theorem. These results provide a sound and fully verified reduction from parametric bit-vector reasoning to natural number arithmetic.

4.10 Verifying Error Correction Coding Algorithms using Symbolic Trajectory Evaluation

Ramit Das (Cadence Design Systems India, IN)

We present a use case of datapath/algorithmic verification in the context of Error Correction Algorithms for Hardware Systems. The lattice-based symbolic simulation methodology of Symbolic Trajectory Evaluation (STE) comes to our rescue for this complex problem. We shall dive into what the tool promises and get a peek into industrial challenges in the domain of datapath verification.

4.11 Incorrectness Proofs with Underapproximation Invariants

Gourav Takhar (IIT Kanpur, IN)

Incorrectness logic provides a theoretical framework to reason about program faults. While incorrectness logic has been immensely successful at explaining bug-finding tools, automating incorrectness logic, especially its backwards variant rule, has remained challenging. The tools based on incorrectness logic, instead, analyze bounded models of programs using a loop unrolling rule that unrolls loops to finite bounds. While such analysis via loop unrolling is quite effective for shallow bugs, discovering deep bugs remains a challenge. In this work, we propose a new proof system, I^3 , that, instead of requiring backwards variants, allows partial incorrectness proofs with a new kind of invariant, the underapproximation invariant. Unlike regular invariants that must hold on all program traces, underapproximation invariants are required to hold only on a subset of traces. We also show that, under certain sufficient conditions, a partial incorrectness proof does guarantee total incorrectness. Given the success of invariant inference in automating partial correctness proofs with Floyd-Hoare logic, we believe that I^3 should also be easier to automate. We implement our proof system within a prover, $I^3\text{Prove}$, that is able to automatically infer suitable underapproximation invariants en route to incorrectness proofs. We compare $I^3\text{Prove}$ against leading bug-finding tools from the SV-COMP23 competition, viz. UAutomizer, CPAChecker, and Veriabs. We also compare it with a recent proposal, Banal, that uses abstract interpretation to reason about incorrectness claims. On our benchmark suite of 106 incorrect programs, $I^3\text{Prove}$ is able to prove the incorrectness claims in 97 programs, while the best of the baseline tools is only able to solve 69 instances.

4.12 Formal Verification of End-to-End Verifiable Election Protocols

Subodh Sharma (IIT Delhi, IN)

End-to-end verifiable (E2E-V) election protocols use cryptographic techniques to provide strong guarantees of vote integrity, privacy, and transparency. However, the complexity of these protocols makes rigorous validation essential. This talk explores how formal methods can be used to specify and verify key election properties, including ballot secrecy, cast-as-intended, recorded-as-cast, and tallied-as-recorded guarantees. We will survey verification approaches based on model checking, theorem proving, and automated protocol analysis, and discuss lessons learned from the formal analysis of real-world voting systems. The talk concludes with emerging challenges at the intersection of cryptography, formal verification, and trustworthy digital democracy.

4.13 Edit Distance of Finite State Transducers

Aiswarya C. (Chennai Mathematical Institute, IN, aiswarya@cmi.ac.in)

Joint work of Aiswarya, C.; Manuel, Amaldev; Sunny, Saina

Main reference Aiswarya C., Amaldev Manuel, Saina Sunny. Edit Distance of Finite State Transducers. ICALP 2024.

We lift metrics over words to metrics over word-to-word transductions, by defining the distance between two transductions as the supremum of the distances of their respective outputs over all inputs. This allows us to compare transducers beyond equivalence. Two transducers are close (resp. k -close) with respect to a metric if their distance is finite (resp. at most k). Over integer-valued metrics computing the distance between transducers is equivalent to deciding the closeness and k -closeness problems. For common integer-valued edit distances such as Hamming, transposition, conjugacy and Levenshtein family of distances, we show that the closeness and the k -closeness problems are decidable for functional transducers. Hence, the distance with respect to these metrics is also computable. Finally, we relate the notion of distance between functions to the notions of diameter of a relation and index of a relation in another. We show that computing edit distance between functional transducers is equivalent to computing diameter of a rational relation and both are a specific instance of the index problem of rational relations.

4.14 Proof Systems for QBF Synthesis: Extracting Skolem and Herbrand Functions

Supratik Chakraborty (IIT Bombay, IN, supratik@cse.iitb.ac.in)

Joint work of Chakraborty, Supratik; Akshay, S.; Beyersdorff, Olaf; Kasche, Lea; Mahajan, Meena; Spachmann, Luc Nicolas

Main reference To be presented at SAT 2026.

Strategy extraction in QBF proof systems usually attempts to extract winning strategies from valid proofs. However, an alternative (and arguably more powerful) view is to extract Skolem/Herbrand functions, or equivalently synthesis of the game values at all intermediate points. In this talk, we investigate the existence and properties of such proof systems from which one can extract Skolem and Herbrand functions. We propose such a proof system for

QBF, which we show is sound and complete, and from which extraction of Skolem/Herbrand functions can be performed, and game values computed, in polynomial time. We also show that this system is optimal among all proof systems that allow efficient extraction of Skolem/Herbrand functions. We provide conditional lower bound results for our new proof system and compare it to several existing/standard proof systems for QBF that have been studied in the literature, showing interesting orthogonality results. Finally, we provide a compilation algorithm that takes an arbitrary QBF and synthesizes a proof in our system, from which Skolem and Herbrand functions can be easily computed.

4.15 Generalized Timed Automata

B. Srivathsan (Chennai Mathematical Institute, IN, sri@cmi.ac.in)

Joint work of Srivathsan, B.; Akshay, S.; Contractor, Prerak; Gastin, Paul; Govind, R.; Joshi, Aniruddha

Timed automata are a widely adopted formalism for modeling real-time systems, using clocks to enforce timing constraints on transitions. In this talk, we present the idea of “future clocks”, which are in some sense dual to ordinary clocks. Future clocks are like timers: they predict the time to a future event, and remain active until they time out. “Generalized timed automata” allow both clocks and future clocks. In the talk, we will introduce this model and describe some of its properties. We will then demonstrate their usefulness in logic-to-automata translations.

4.16 Sure-Almost-Sure and Sure-Limit-Sure Window Mean Payoff in Markov Decision Processes

Pranshu Gaba (TIFR Mumbai, IN)

Given rationals α and β , the sure-almost-sure problem for a quantitative objective φ in a Markov decision process (MDP) asks if one can simultaneously ensure that all outcomes of the MDP have φ -value at least α (i.e., sure α satisfaction), and with probability 1 the outcome has φ -value at least β (i.e., almost-sure β satisfaction). Moreover, if simultaneous satisfaction of objectives is possible, then one would also like to construct a strategy that achieves this. Even if both sure satisfaction and almost-sure satisfaction for an objective are known, combining the two is often non-trivial and requires novel techniques and approaches. In this talk, we look at the sure-almost-sure problem for window mean-payoff objectives. The window mean-payoff objective strengthens the standard mean-payoff objective by requiring that the average payoff from each point in the play become greater than the threshold in at most ℓ steps. For the window mean-payoff objective, we show that while the computational complexity of the sure-almost-sure problem matches that of sure satisfaction and almost-sure satisfaction, the memory requirement of winning strategies for sure-almost-sure satisfaction is greater than that of both sure and almost-sure satisfaction.

4.17 Decision Tree Ensembles: A Sweet Spot between Machine Learning and Formal Methods

S. Akshay (IIT Bombay, IN, akshayss@cse.iitb.ac.in)

Decision trees and their ensembles are among the most widely used machine learning models, particularly for tabular data. Their discrete and structured nature also makes them attractive targets for formal reasoning. In this talk, we will survey the state of the art in the verification of decision tree ensembles and discuss recent advances in the area. We will see how formal reasoning and symbolic techniques enable rigorous analysis of decision tree ensembles, with applications to robustness, explainability, and sensitivity, and conclude by discussing some recent developments in sensitivity verification and quantification.

4.18 Understanding the IC3 verification algorithm

Deepak D'Souza (Indian Institute of Science, IN, deepakd@iisc.ac.in)

The IC3 algorithm for verifying safety properties was proposed by Aaron Bradley in 2011. Since then the approach has found widespread use across domains including hardware, software, and discrete-time systems. In this talk we describe the basic algorithm in the setting of boolean systems, its extension to software verification, and try to bring out the reasons for its effectiveness.

4.19 Scaling Probabilistic Model Checking with Model Counting

Arijit Shaw (Chennai Mathematical Institute, IN)

The problem of finite-horizon reachability in discrete-time Markov chains (DTMCs) is to determine the probability of reaching a designated target state within h steps. Finite-horizon reachability is a fundamental problem in probabilistic model checking with a wide range of applications. Despite sustained progress, the scalability of probabilistic and statistical model checkers is severely impacted by state-space explosion, particularly for systems with parallel processes, thereby preventing their adoption in application domains requiring verification of large-scale concurrent systems. In this talk, we address this scalability bottleneck: we present a novel reduction from probabilistic model checking to weighted model counting, together with an approximate weighted model counter tailored to handle these instances efficiently. The resulting tool solves instances with up to $40\times$ more parallel processes than state-of-the-art probabilistic model checkers.

4.20 Circuit Complexity of Regular Languages

Amaldev Manuel (IIT Goa, IN, amal@iitgoa.ac.in)

Joint work of Manuel, Amaldev; Göller, Stefan

Many natural classes of languages defined via logic, automata, or finite semigroups form varieties, i.e., they are closed under Boolean operations, inverse morphisms, and quotients by

words. This connection extends to Boolean circuits if inverse length-multiplying morphisms replace inverse morphisms; the resulting classes are called *lm-varieties* (Straubing, 2002). An Eilenberg-type correspondence holds: *lm-varieties* of languages correspond to classes of syntactic morphisms satisfying suitable HSP conditions. As an example, a strengthening of the Furst–Saxe–Sipser theorem (Parity is not in AC^0) characterises the regular languages computed by constant-depth polynomial-size circuits as exactly the *lm-variety* QA , i.e., those whose syntactic morphisms have aperiodic stable semigroups (Barrington et al. 1992). We use the framework of *lm-varieties* to give an effective classification of the circuit complexity of regular languages.

4.21 Set Automata and Limits of Decidability of Two-Variable Logic on Data Words

Rishal S. P. (Sarva Labs, IN)

Joint work of Rishal, S. P.; Guha, Shibashis; Manuel, Amaldev

We extend the two-variable logic on data words with guarded regular binary predicates of the form $L(x, y)$ that is true if positions x and y have the same data value and the factor strictly between x and y is in the regular language L . We characterise the class of monoids for which the extension of the two-variable logic with guarded predicates recognised by the monoid is decidable, namely the class of idempotent monoids whose two-sided ideals are linearly ordered. For this, we introduce an automata formalism, *set automata*, that has an undecidable emptiness problem. A *set automaton* is a finite state automaton with a fixed number of sets that can store data values. During the run, the automaton can add or remove the current data value from the sets as well as assign a union of sets to each set. The latter kind of updates can be represented as an element of a semigroup of relations on the sets. The acceptance of a run is determined by the state as well as the sets in which the data values are present at the end of the run. We identify a subclass of *set automata* called *ordered quasi-normal set automata* that has a decidable emptiness problem by reduction to the emptiness problem of ordered multicounter automata. We show that the two-variable logic extended with guarded regular predicates recognised by a semigroup S is expressively equivalent to a *quasi-normal set automaton* with the semigroup of relations S . In particular, if S is a linear band monoid then the resulting automaton is ordered, and the decidability result follows.

4.22 Regular Expressions over Countable Words

Sreejith A. V. (IIT Palakkad, IN, sreejithav@iitpkd.ac.in)

We investigate the expressive power of regular expressions for languages of countable words and establish their expressive equivalence with logical and algebraic characterizations. Our goal is to extend the classical theory of regular languages, defined over finite words and characterized by automata, monadic second-order logic, and regular expressions, to the setting of countable words.

4.23 Synthesizing POMDP Policies: Sampling meets Model-checking via Learning

Sayan Mukherjee (IIT Bombay, IN)

Joint work of Mukherjee, Sayan; Chakraborty, Debraj; Majumdar, Anirban; Mathew, Prince; Raskin, Jean-François

Main reference To appear in CAV 2026.

Partially Observable Markov Decision Processes (POMDPs) are the standard framework for decision-making under uncertainty. While sampling-based methods scale well, they lack formal correctness guarantees, making them unsuitable for safety-critical applications. Conversely, formal synthesis techniques provide correctness-by-construction but often struggle with scalability, as general POMDP synthesis is undecidable. To bridge this gap, we propose a synthesis framework that integrates sampling, automata learning, and model-checking. Inspired by Angluin’s L^* algorithm, our approach utilizes sampling as a membership oracle and model-checking as an equivalence oracle. This enables the synthesis of finite-state controllers with formal guarantees, provided the sampling-induced policy is regular. We establish a relative completeness result for this framework. Experimental results from our prototypical implementation demonstrate that this method successfully solves threshold-safety problems that remain challenging for existing formal synthesis tools. We believe our algorithm serves as a valuable component in a portfolio approach to tackling the inherent difficulty of POMDP synthesis problems.

4.24 Deciding Effectively Propositional Logic with Function Symbols

Abhisekh Sankaran (TCS Research, IN)

Effectively Propositional Logic (EPR), also called the Bernays–Schönfinkel–Ramsey class (BSR) or the Σ_2 fragment of first-order logic (FO), is a widely studied subclass of FO. It has been used across a variety of areas, including the verification of programs and protocols, program synthesis, finite model theory, and the structure theory of graphs. This logic is known to be decidable when the vocabulary is relational; in particular it enjoys the small model property.

In two recent papers [1] and [2], the authors introduced an extension of EPR in a multi-sorted setting, called the Ordered Self-Cycle fragment (OSC), in the context of verifying distributed consensus protocols and heap-manipulating programs. Under Skolemization and a few other syntactic transformations, this logic is equisatisfiable with a very simple uni-sorted fragment of it, denoted OSC^* , that consists of universal FO formulas with a single universal quantifier, finitely many function and relation symbols that are at most unary, and one distinguished binary relation symbol apart from equality.

The authors show that over ordered domains (where the binary relation is interpreted as a linear order), OSC^* , which contains infinity axioms, enjoys the *small symbolic model property*, where symbolic models are finite presentations of certain classes of infinite structures, and “small” means computably bounded. As a consequence, OSC^* , and hence OSC, is decidable for satisfiability over (ordered) arbitrary structures, well-founded structures, and finite structures. These results are particularly interesting as they contrast with the known undecidability, over (unrestricted) arbitrary and finite structures, of the class of EPR formulae with a single universal quantifier and two unary function symbols as the only non-logical symbols apart from equality.

We will survey some of these results in the talk.

References

- 1 Elad, N., Padon, O., & Shoham, S. (2024). An infinite needle in a finite haystack: Finding infinite counter-models in deductive verification. *Proceedings of the ACM on Programming Languages*, 8(POPL), 970–1000.
- 2 Elad, N., & Shoham, S. (2025). Axe'Em: Eliminating Spurious States with Induction Axioms. *Proceedings of the ACM on Programming Languages*, 9(POPL), 479–508.

Participants

- Chaitanya Agarwal
New York University
- Smayan Agarwal
Ashoka University
- Anvit Aggarwal
IIT Gandhinagar
- Aarav Akali
Krea University
- S. Akshay
IIT Bombay
- C. Aiswarya
Chennai Mathematical Institute
- Ayush Anand
IIT Bhubaneswar
- Shabana A. T.
NIT Calicut
- Akhilesh Balaji
Ashoka University
- A. Baskar
BITS Pilani, Goa
- Ashkabsha C.
NIT Calicut
- Raseek C.
Government Engineering College,
Thrissur
- Supratik Chakraborty
IIT Bombay
- Rajiv Dalal
VIT Vellore
- Ramit Das
Cadence Design Systems India
Pvt Ltd
- Abhishek De
Krea University
- Soumya Dey
Krea University
- Avijit Dhangar
VIT Vellore
- Deepak D'Souza
Indian Institute of Science
- Pranshu Gaba
TIFR Mumbai
- Siddhartha Gadgil
Indian Institute of Science
- Vedant Gautam
Ashoka University
- Alphy George
NIT Calicut
- Sujata Ghosh
Indian Statistical Institute
- Vaani Goenka
Ashoka University
- Govind R.
IMSc
- Shibashis Guha
TIFR Mumbai
- Nidhi Gunjal
VIT Vellore
- Ashutosh Gupta
IIT Bombay
- Inzemamul Haque
Krea University
- Sarah Idikula
Krea University
- Gabriel James
Government Engineering College,
Thrissur
- Varsha Jarali
SETS Chennai
- Amrit M. Joseph
Indian Institute of Science
- Mridul Joy
Government Engineering College,
Thrissur
- Vivek Rajesh Joshi
IISER Pune
- Neeraj Krishnan K.
IIT Goa
- Piyush Kurur
IIT Palakkad
- Mithilesh Kumar
Krea University
- Anirban Majumdar
TIFR Mumbai
- Satpreet Makhija
Ashoka University
- Amaldev Manuel
IIT Goa
- Subhadip Masanta
Indian Statistical Institute,
Kolkata
- Ravindra Metta
TCS Research
- Sahil Mhaskar
Chennai Mathematical Institute
- Aniket Mishra
IIT Gandhinagar
- Ashish Mishra
IIT Hyderabad
- Bhumika Mittal
Georgia Institute of Technology
- Rishov Mondal
Indian AI Research Organisation
- Sayan Mukherjee
IIT Bombay
- Madhavan Mukund
Chennai Mathematical Institute
- Hurudaya Narasimhan
Krea University
- Chandralekha P.
TIFR Mumbai
- Ezudheen P.
Government Engineering College,
Thrissur
- Shashi Kant Pandey
SETS Chennai
- Shashank Pandey
IISER Tirupati
- Naman Paul
TCS Research
- T. V. H. Prathamesh
Krea University
- Ayush Raj
Government Engineering College,
Thrissur
- Shweta Rajiv
The University of Iowa
- Sheerazuddin S.
NIT Calicut
- Rishal S. P.
Sarva Labs
- Saptarshi Sahoo
Indian Statistical Institute,
Chennai
- Kshitij Salunke
IISER Mohali
- Stanly Samuel
Collins Aerospace
- Abhisekh Sankaran
TCS Research
- Subodh Sharma
IIT Delhi
- Suhana Sharma
TCS Research
- Sarang Shastry
BITS Pilani, K. K. Birla Goa
Campus
- Arijit Shaw
Chennai Mathematical Institute
- Sreejith A. V.
IIT Palakkad
- B. Srivathsan
Chennai Mathematical Institute
- S. P. Suresh
Chennai Mathematical Institute
- Srikumar K. Subramanian
Krea University
- Gourav Takhar
IIT Kanpur
- Yusuf Tahir
Krea University
- Aalok Thakkar
Ashoka University
- Abhishek Vijay Uppar
Indian Institute of Science
- Sagar Verma
TCS Research
- Rishi Vyas
Krea University
- Anand Yeolekar
TCS Research